

Национальный исследовательский университет ИТМО
Факультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по практическому заданию №3
по курсу «Языки программирования»
Вариант: 4

Выполнил студент группы Р4119 _____ Н.Ю. Ковалев
(подпись)

Руководитель _____ Ю. Д. Кореньков
(подпись)

Санкт-Петербург
2026

Цели

- Получить линейный код из CFG-представления подпрограмм, описать виртуальную машину (регистр, инструкции, память) и вывести мнемонический листинг, готовый к запуску на эмуляторе.
- Демонстрировать работу генератора через командный интерфейс, который перепускает CFG, добавляет эпилоги/прологи и визуально/текстово фиксирует результат в `out.asm`.

Задачи

1. Описать VM-структуры для встраивания инструкций и литералов: регистры `r0..r5`, стековый `SP/BP`, кодовая секция (`start, call ..., hlt`) и данные (символы `out.asm`).
2. Создать модуль генерации линейного кода (`Lab2/linear_code.c/linear_code.h`): структуры `ProgramImage`, `CodeBlock`, `DataItem`, `FlowOperation`, `ExprInfo` и `EmitContext` формируют выпуск инструкций для каждого базового блока/операции.
3. Обновить тестовую программу (`lab2_cfg`) для поступления CFG и вывода собранного листинга (`program_image_write_asm`), а также сохранить DOT/CFG-информацию и готовый `out.asm`.
4. Построить виртуальные инструкции (`mov, add, li, cmp, jmp, call, ret, outb/outd`) для каждой операции и трассы переходов в CFG, включая обработку вызовов `send_byte`, ветвлений и возвращений.
5. Представить результаты тестирования (скрипт сборки, генерация DOT/ASM, проверенный пример) и проиллюстрировать, как `exmpl.txt`/другие файлы превращаются в линейный код.

Описание работы

Генератор линейного кода использует структуру `ProgramImage`, которая сочетает `CodeBlock` (заголовки функций) и `DataItem` (константы/литералы). `lab2_cfg` получает CFG из `Lab2/flow.c`, вызывает `program_image_add_function_from_cfg` и, после обработки всех функций, записывает файл `out.asm` через `program_image_write_asm`. Каждая функция получает ярлык (например, `calc_txt_main`), пролог/эпилог (`PUSH BP, MOV BP, SP, RET`) и инструкции для операций, извлечённых из `FlowOperation`. Инструкции выводятся в стиле виртуальной архитектуры: `li` для загрузки констант, `mov/add/sub/mul/div` для арифметики, аппаратные вызовы `outb/outd` для `send_byte`, и `jmp/jz/jnz` для ветви.

Архитектура VM (по `architecture/twoaddr_dyn64_v10.target.pdsl`)

1. Регистры и представления.

```
storage ip [32];
storage sp [32];
storage r0st [64];
view r0 = r0st;
```

IP/SP задают поток исполнения, `r0..r7` хранят тегированные значения; `EmitContext` копирует эту модель, привязывая переменные к `r0..r3`.

2. Стек и вызовы.

```
instruction push = { ... } { sp = (sp - 8) & 0xffff; stackMem:8[sp] = src; ip = ip + 4; }
```

`program_image_add_function_from_cfg` вставляет эти шаги в пролог/эпилог, моделируя стековые кадры вызовов.

3. Арифметика/логика.

```
instruction add = { ... dst, src ... } { dst = dst + src; }
instruction cmp = { ... } { zf = (dst == src); sf = (dst < src); }
```

`emit_binary_expr/binary_kind_instr` генерирует `add/sub/mul/div`, а `emit_branch` использует `cmp` чтобы установить `zf/sf`.

4. Ветвления.

```
instruction jz = { ... imm16 } { if zf != 0 then ip = imm16; }
```

emit_branch выпускает jz/jnz/jmp согласно true/false меткам, повторяя PDsl-описание.

5. IO и trap.

```
instruction outd = { ... } { rout = src; }
instruction inb = { ... } { dst = rin & mask; }
instruction hlt = { ... } { // stop }
```

emit_builtin_call транслирует send_byte/print_int в outb/outd, program_image_write_asm завершает out.asm hlt, что соответствует описанию VM.

Аспекты реализации

- 1. **Регистровый контекст (EmitContext).** Модуль распределяет регистры r0..r3 под переменные, а ensure_var_reg при необходимости добавляет новую привязку.

```
static const char *ensure_var_reg(EmitContext *ctx, const char *name) {
    if (!name) return scratch_reg;
    const char *existing = lookup_var_reg(ctx, name);
    if (existing) return existing;
    ...
    ctx->entries[ctx->n].name = strdup(name);
    ctx->entries[ctx->n].reg = reg;
    ctx->n++;
    return reg;
}
```

- 2. **Формирование инструкций из выражений.** emit_expr_value парсит Nop(Identifier), Nop(Literal), Call(...) и BinaryOp(...), подставляя li, mov и add/sub/mul/div в битовом контексте.

```
if (strstr(trimmed, "Nop(Literal)") && strstr(trimmed, "[const:")) {
    emit_instruction(b, "    li %s, %lld", target, value);
    return;
}
if (strstr(trimmed, "Call(")) {
    emit_call_expr(b, ctx, trimmed, target);
    return;
}
if (strstr(trimmed, "BinaryOp(")) {
    emit_binary_expr(b, ctx, trimmed, target);
    return;
}
```

- 3. **Обработка операций (emit_flow_operation).** Каждая FlowOperation (присваивание, вызов, возврат) трансформируется в инструкции с комментариями и спец-вызовами outb/outd для встроенных print_char, print_int.

```
switch (op->kind) {
    case FLOW_OP_ASSIGN:
        emit_binary_assignment(...);
        break;
    case FLOW_OP_CALL:
        emit_comment(b, "CALL ...");
        if (emit_builtin_call(...)) break;
        ... emit_instruction(b, "    call %s", target);
        break;
}
```

- 4. **Генерация веток при обходе CFG.** emit_branch читает cfg->nodes[node_id].succ и выводит jmp/jz/jnz в зависимости от меток true/false и CondKind.

```
if (cond_op && cond_op->cond_kind != CONDK_UNKNOWN) {
    emit_instruction(b, "    cmp %s, %s", left_reg, right_reg);
    emit_instruction(b, "    %s %s", cond_false_instr(cond_op->cond_kind), labels[false_id]);
    if (true_id != -1 && true_id != next) emit_instruction(b, "    jmp %s", labels[true_id]);
}
```

- 5. **Сборка образа и запись.** program_image_add_function_from_cfg делает DFS по CFG, проставляет метки, добавляет блоки к ProgramImage, а program_image_write_asm выводит заголовок (ldsp, call,

hlt) и все функции/данные в out.asm.

```
fprintf(out, "start:\n ldsp 0xffff0\n ldbp 0xffff0\n call %s\n hlt\n", entry_label);
for (int i=0;i<img->n_blocks;i++) {
    fprintf(out, "; function %s\n", b->name);
    for (int j=0;j<b->lines.n_lines;j++) fprintf(out, "%s\n", b->lines.lines[j]);
}
```

Результаты

- Получен ассемблерный листинг out.asm с секцией code, загрузкой SP/BP, вызовом start, инструкциями li/mov/add/..., call, ret, jmp, outb/outd; плюсом данные (.data) с литералами.
- Собраны описания функций (CodeBlock) и данных (DataItem), включённые в ProgramImage; регистры r0..r5, BP, SP и scratch_reg реализуют модель памяти/регистров.
- Вызовы send_byte преобразованы в outb/outd, встроенные арифметические операции — в add/sub/mul/div, а контрольные переходы — в jmp, jz, jnz, обеспечивая соответствие заданным категориям инструкций.

Результаты тестирования

Lab2/run_tests.sh компилирует lab2_cfg, запускает Lab2/generate_cfgs.sh, проверяет наличие Lab2/out/*.callgraph.dot, all_functions.dot и выводит файлы DOT/ASM. Скрипт послужил проверкой того, что линейный код генерируется для каждого CFG и сохраняется в Lab2/out + out.asm.

Выводы

- Построен модуль линейного генератора, работающий с CFG и итеративно создающий инструкции, комментарии и ветви; виртуальная машина задана регистрами и инструкциями (li, mov, cmp, jmp, call, ret, outb/outd).
- CLI lab2_cfg + Lab2/run_tests.sh демонстрируют сквозную цепочку: синтаксический анализ → CFG → линейный код → листинг out.asm / DOT / call-graph.
- Полученные артефакты (Lab2/out/*.dot, out.asm, Lab2/out/all_functions.dot, Lab2/out/*.callgraph.*) покрывают инструкции перемещения, арифметики, ветвлений и ввода-вывода, удовлетворяя требованиям третьей лабораторной.